

PROJET

CAHIER DES CHARGES

Lina BERROUG, Jessica KAREGA (Binôme sans machine virtuelle)

INTRODUCTION

L'objectif du projet est de créer une base de données d'une pharmacie pour gérer les clients (anonymes et enregistrés), les ventes avec ou sans ordonnances de médicaments, les commandes auprès des fournisseurs et les facturations et enfin avoir un suivi de ces procédures dans un but de traçabilité des produits vendus ou commandés et sécurité et faciliter la prise de décision.

Informations générales

Clients : Les clients effectuent des achats de médicaments avec ou sans ordonnance auprès d'un pharmacien, (ils peuvent être couverts ou non par une mutuelle) pour chaque client on souhaite avoir les informations telles que son nom, son prénom son adresse et son contact (numéro de téléphone).

Pharmaciens : Les pharmaciens de l'officine délivrent/vendent les médicaments aux clients, chaque pharmacien est identifié par un numéro NSSI et au même titre qu'un client ils possèdent un nom, son prénom, son adresse et son contact.

Médecins : Les médecins prescripteurs : les médecins prescrivent des ordonnances aux clients ils sont caractérisés par un numéro NSSI, un nom, son prénom, son adresse et son contact.

Les fournisseurs : Les fournisseurs livrent des lots de médicaments à la pharmacie à l'issue d'une commande, ils sont caractérisés par un identifiant, un mail, une adresse et numéro de téléphone.

Les médicaments : Les médicaments peuvent être achetés par des clients et sont contenus dans des lots. Ces derniers sont caractérisés par un code CIP, un nom, un prix public (prix à payer sans ordonnance), une description, un statut de vente (est-ce qu'ils peuvent être vendu ou pas sans ordonnance) et le laboratoire où ils ont été conçus.

La couverture santé des clients : Pour chaque client qui dispose d'une ordonnance, le taux de remboursement ne sera pas le même. La couverture santé est caractérisé par un numéro AMC et un taux de remboursement

Les lots de médicaments : Dans notre pharmacie, les médicaments sont organisés en lots. Chaque lot contient un seul type de médicament qui ont tous la même date de fabrication et la même date de péremption. Ils sont aussi caractérisés par un numéro de lot unique et une quantité. Les lots sont commandés par la personne chargée des stocks et sont fournis par les fournisseurs.

Les fournisseurs : Reçoivent la commande faite par la pharmacie et livrent les lots commandés. Pour chaque fournisseur, on souhaite connaître leur mail, numéro de téléphone, leur adresse et la ville où ils se situent.

Les commandes passées aux fournisseurs : Les commandes sont passées par le gestionnaire des commandes en fonction du stock restant. Chaque commande doit posséder un identifiant, une date de commande, un prix, et un statut (fournis ou pas encore)

Les ordonnances remises par les clients: Une ordonnance appartient à un client et a été émise par un médecin. On doit connaître pour chaque ordonnance son identifiant, la date de prescription et la date de péremption.

Les lignes d'ordonnance : Une ligne d'ordonnance est comprise dans une ordonnance prescrite à un client, elle contient un seul médicament. Elle se caractérise par un

Les ventes réalisées à la pharmacie pour le client sont caractérisées par un identifiant, une date de vente et un prix final (c'est-à-dire la somme du prix de chaque médicament acheté).

Les lignes de vente : Chaque achat effectué par un client peut inclure différents médicaments provenant de lots distincts. Tous les médicaments associés à un achat sont enregistrés dans les lignes de vente. Chaque ligne de vente est caractérisée par un identifiant unique, la quantité du médicament acheté et son prix (avec ou sans remboursement).

Modélisation

Voir Annexes

Contraintes d'intégrités

I. Optimisation des Ventes

Calcule le prix final d'un médicament en appliquant le taux de remboursement de la mutuelle du client. Si aucune mutuelle n'est trouvée, le prix public total est appliqué. (= Trigger 2)

```
CREATE OR REPLACE TRIGGER LIGNEVENTE_CALCUL_PRIX
BEFORE INSERT OR UPDATE ON LIGNEVENTE
FOR EACH ROW
FOLLOWS TRG_LIGNEVENTE_1_AUTO_LOT
DECLARE
    v_prix_pub NUMBER(8,2);
    v_taux      NUMBER(5,2) := 0;
BEGIN
    IF :NEW.numero_de_lot IS NOT NULL THEN
        -- 1. Récupération du prix public
        BEGIN
            SELECT M.prix_public INTO v_prix_pub
            FROM MEDICAMENT M
            JOIN LOT L ON L.code_cip = M.code_cip
            WHERE L.num_lot = :NEW.numero_de_lot;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN v_prix_pub := 0;
        END;

        -- 2. Récupération du taux de remboursement
        BEGIN
            SELECT COUV.taux_de_rembursement INTO v_taux
            FROM VENTE V
            JOIN CLIENT C ON V.id_Client = C.NSSI
            JOIN COUVERTURE COUV ON C.Nom_mutuelle = COUV.Nom_mutuelle
            WHERE V.id_Vente = :NEW.id_Vente;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN v_taux := 0;
        END;

        -- 3. Calcul du prix final
        :NEW.prix_après_rembursement := (:NEW.quantité_vendu * v_prix_pub) * (1 - (v_taux / 100));
    END IF;
END;
```

Sélection du lot selon la date de péremption : Si le pharmacien ne saisit pas de lot, le trigger choisit automatiquement celui qui périm le plus tôt via l'ordonnance. Si le pharmacien saisit manuellement un lot, le système vérifie s'il existe un lot plus ancien et le remplace pour éviter le gaspillage. (= Trigger 1)

```

CREATE OR REPLACE TRIGGER TRG_LIGNEVENTE_1_AUTO_LOT
BEFORE INSERT OR UPDATE ON LIGNEVENTE
FOR EACH ROW
DECLARE
    v_cip NUMBER;
    v_lot_p NUMBER;
    v_date_p DATE;
    v_date_saisie DATE;
BEGIN
    IF :NEW.numero_de_lot IS NULL AND :NEW.id_ordonnance IS NOT NULL THEN
        SELECT id_medicament INTO v_cip
        FROM LIGNEORDONNANCE
        WHERE id_ordonnance = :NEW.id_ordonnance
        AND ROWNUM = 1;

    ELSIF :NEW.numero_de_lot IS NOT NULL THEN
        SELECT code_cip, Date_Peremption INTO v_cip, v_date_saisie
        FROM LOT
        WHERE num_lot = :NEW.numero_de_lot;

    ELSE
        RAISE_APPLICATION_ERROR(-20010, 'saisir soit un lot ou une ordonnance');
    END IF;

    SELECT num_lot, Date_Peremption INTO v_lot_p, v_date_p
    FROM (
        SELECT num_lot, Date_Peremption
        FROM LOT
        WHERE code_cip = v_cip
        AND Quantite >= :NEW.quantite_vendu
        ORDER BY Date_Peremption ASC
    ) WHERE ROWNUM = 1;

    IF v_date_saisie > v_date_p THEN
        :NEW.numero_de_lot := v_lot_p;
    END IF;

```

Calcul du Total de la Facture : Met à jour automatiquement le montant total (prix final) de la vente à chaque fois qu'une ligne de médicament est ajoutée, modifiée ou supprimée. (= Trigger 4)

```

CREATE OR REPLACE TRIGGER Calcule_Prix_Vente
AFTER INSERT OR UPDATE OR DELETE ON LIGNEVENTE
BEGIN
    UPDATE VENTE
    SET prixfinal = (
        SELECT NVL(SUM(prix_après_remboursement),0)
        FROM LIGNEVENTE
        WHERE LIGNEVENTE.id_vente = VENTE.id_vente
    )
    WHERE id_vente IN (SELECT DISTINCT id_vente FROM LIGNEVENTE);
END;
/

```

II. Gestion des Stocks

Mise à jour Automatique des Stocks : Déduit immédiatement la quantité vendue du stock disponible dans la table des lots après chaque vente. (= Trigger 3)

```
CREATE OR REPLACE TRIGGER TRG_LIGNEVENTE_3_MAJ_STOCK
AFTER INSERT ON LIGNEVENTE
FOR EACH ROW
DECLARE
    v_stock_actuel NUMBER;
BEGIN
    -- 1. Vérification du stock restant
    SELECT Quantite INTO v_stock_actuel FROM LOT WHERE num_lot = :NEW.numero_de_lot;

    IF v_stock_actuel < :NEW.quantité_vendu THEN
        RAISE_APPLICATION_ERROR(-20003, 'ERREUR STOCK : Quantité insuffisante (Dispo: ' || v_stock_actuel || ').');
    ELSE
        -- 2. Mise à jour
        UPDATE LOT
        SET Quantite = Quantite - :NEW.quantité_vendu
        WHERE num_lot = :NEW.numero_de_lot;
    END IF;
END;
/
```

Contrôle de Conformité des livraisons : Lors de la réception de nouvelles marchandises, vérifie que la quantité livrée par le fournisseur correspond exactement à la quantité commandée. (= Trigger 5)

```
CREATE OR REPLACE TRIGGER verif_livraison_conforme
BEFORE INSERT ON LOT
FOR EACH ROW
-- FONCTIONNE
DECLARE
    v_quantite_attendue NUMBER;
BEGIN
    SELECT Quantite
    INTO v_quantite_attendue
    FROM COMMANDE
    WHERE id_Commande = :NEW.id_Commande;

    IF :NEW.Quantite != v_quantite_attendue THEN
        DBMS_OUTPUT.PUT_LINE('Attention : la quantité du lot ne correspond pas à la commande.');
```

```
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20009, 'ERREUR : Le numéro de commande ' || :NEW.id_Commande || ' n''existe pas.');
```

```
END;
/
```

III. Sécurité et Conformité Légale

Vérification de la Validité de l'Ordonnance : Interdit strictement la vente si la date de péremption de l'ordonnance est dépassée au moment de la transaction. (= Trigger 7)

```
CREATE OR REPLACE TRIGGER VALIDITE_ORDONNANCE?
BEFORE INSERT ON LIGNEVENTE
FOR EACH ROW
DECLARE
    DP DATE;

BEGIN
    SELECT date_peremption INTO DP
    FROM ORDONNANCE
    WHERE id_ordonnance = :NEW.id_ordonnance;

    IF DP < SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20001, 'ordonnance a expiré ');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        NULL;
END;
/
```

Contrôle de Sur-délivrance : empêche le pharmacien de vendre une quantité de médicament supérieure à celle prescrite par le médecin sur l'ordonnance. (= Trigger 6)

```
CREATE OR REPLACE TRIGGER verif_surdelivrance
BEFORE INSERT ON lignevente -- FONCTIONNE
FOR EACH ROW
DECLARE
    v_qt_prescrire NUMBER;
BEGIN
    -- On récupère la quantité prescrite pour **la première ligne** de l'ordonnance
    SELECT qt_delivre
    INTO v_qt_prescrire
    FROM ligneordonnance
    WHERE id_ordonnance = :NEW.id_ordonnance
    AND ROWNUM = 1;

    IF :NEW.quantité_vendu > v_qt_prescrire THEN
        RAISE_APPLICATION_ERROR(-20002, 'Erreur : la quantité vendue dépasse la quantité prescrite pour ce médicament. ');
    END IF;
END;
/
```

IV. Protection des Données et Historique

Modification des des Ventes Passées interdites en pratique : Interdit toute modification ou suppression d'une ligne de vente dès que la date de la transaction est passée, garantissant ainsi l'intégrité de la comptabilité et de la traçabilité. (= Trigger 8)

```
CREATE OR REPLACE TRIGGER VERROUILLAGE_LIGNE
BEFORE UPDATE OR DELETE ON LIGNEVENTE
FOR EACH ROW
DECLARE
    D DATE;
BEGIN
    SELECT date_vente INTO D
    FROM VENTE
    WHERE id_Vente = :OLD.id_Vente;

    IF D < TRUNC(SYSDATE) THEN
        RAISE_APPLICATION_ERROR(-20003, 'Modification interdite.');
```

Droits et Vues

Pour garantir la confidentialité des données, l'accès aux tables de base est interdit aux utilisateurs. Des vues ont été définies pour limiter l'accès seulement aux informations nécessaires selon le rôle de l'utilisateur (ex : pharmacien, comptable, gestionnaire des commandes...). Les droits sont accordés uniquement sur ces vues, ce qui permet de protéger les données sensibles et non pertinentes.

- **Vue 1** : Vue affichant les informations détaillées des ordonnances actuellement en cours de traitement, incluant les patients, les médicaments prescrits et les quantités à délivrer

```
create view Stock_risque as
select m.nom
from medicament m
join lot l on l.CODE_CIP = m.code_cip
group by m.nom
having sum(l.Quantite) <= 500;
```

- **Vue 2** : Vue des médicaments dont le stock est faible (c'est à dire inférieur ou égal à 500) et présente un risque de rupture

```
create or replace view infos_ordonnance as
select o.id_ordonnance,
       o.date_prescription as date_ordonnance,
       o.date_de_peremption,
       c.nssi,
       c.nom,
       c.prenom,
       c.nom_mutuelle,
       lo.id_medicament,
       lo.qt_delivre,
       lo.duree_trait,
       lo.date_traitement,
       lo.id_rpps
from ordonnance o
join client c on c.nssi = o.nssi
join ligneordonnance lo on o.id_ordonnance = lo.id_ordonnance
where lo.date_traitement >= sysdate - 2;
```

- **Vue 3** : Vue centralisée du catalogue des médicaments, incluant caractéristiques et disponibilité en stock

```
create view Catalogue_Stock_medicament as
SELECT m.code_cip,
       m.nom,
       m.prix_public,
       m.Categorie,
       m.Statue_Vente,
       m.Laboratoire,
       sum(l.Quantite) as quantite_totale,
       min(l.date_peremption) as date_peremption_proche,
       count(l.num_lot) as nombre_lots
from medicament m
join lot l on l.CODE_CIP = m.code_cip
group by m.code_cip, m.nom, m.prix_public, m.Categorie, m.Statue_Vente, m.Laboratoire;
```

- **Vue 4** : Vue des ventes et du chiffre d'affaires mensuel

```
CREATE OR REPLACE VIEW VUE_CA_MENSUEL AS
SELECT
    TO_CHAR(datevente, 'MM-YYYY') AS MOIS,
    COUNT(id_Vente) AS NB_VENTES,
    SUM(PrixFinal) AS TOTAL_REVENUS
FROM VENTE
GROUP BY TO_CHAR(datevente, 'MM-YYYY')
ORDER BY MIN(datevente);

GRANT SELECT ON VUE_CA_MENSUEL TO COMPTABLE;
```

- **Vue 5** : Vue des dépenses mensuelles des commandes de la pharmacie

```
CREATE OR REPLACE VIEW DEPENSE AS
SELECT TO_CHAR(C.Date_Commande, 'MM-YYYY') AS MOIS, SUM(C.Prix_Commande) AS DEPENSE
FROM COMMANDE C
GROUP BY TO_CHAR(C.Date_Commande, 'MM-YYYY')
ORDER BY MIN(C.Date_Commande);
```

- **Vue 6** : Vue qui permet à un client de consulter son historique personnel de ventes

```
-- Vue 6 : Vue qui permet à un client de consulter son historique personnel de ventes
CREATE OR REPLACE VIEW HISTORIQUE AS
SELECT M.NOM, M.Categorie, LV.quantité_vendu, LV.PRIX_APRÈS_REMBOURSEMENT, V.dateVente
FROM MEDICAMENT M
JOIN LOT L ON l.code_cip = m.code_cip
JOIN LIGNEVENTE LV ON lv.numero_de_lot = l.num_lot
JOIN VENTE V ON V.id_Vente = LV.id_Vente
JOIN CLIENT CL ON cl.NSSI = v.id_Client
WHERE CL.NSSI = USER;
```

- **Vue 7** : Montant total des remboursements dus par chaque mutuelle à la pharmacie

```
CREATE OR REPLACE VIEW Remboursement_Mutuelles AS
SELECT C.Nom_mutuelle, SUM( (M.prix_public * Lv.quantité_vendu) - LV.PRIX_APRÈS_REMBOURSEMENT) as Total_Recouvrer
FROM MEDICAMENT M
JOIN LOT L ON L.CODE_CIP = M.CODE_CIP
JOIN LIGNEVENTE LV ON LV.numero_de_lot = L.num_lot
JOIN VENTE V ON V.id_Vente = LV.id_Vente
JOIN CLIENT CL ON CL.NSSI = V.id_Client
JOIN COUVERTURE C ON C.Nom_mutuelle = CL.Nom_mutuelle
GROUP BY C.Nom_mutuelle;
```

Droits et Vues Pharmacien :

Un pharmacien peut enregistrer un nouveau traitement d'ordonnance et accéder aux informations de l'ordonnance qu'il est en train de traiter. De plus, il peut accéder aux informations du client dont il traite l'ordonnance.

Ensuite, le pharmacien peut créer une nouvelle ligne de vente correspondant à la délivrance des médicaments prescrits.

Il peut également accéder à toutes les informations concernant les lots de médicaments, les médicaments eux-mêmes ainsi que les fournisseurs, afin d'assurer une gestion correcte des stocks et des produits délivrés.

Cependant, il ne peut pas modifier les informations sur un traitement d'ordonnance ou d'une vente qu'il n'a pas réalisé. Il ne peut modifier aucune information dans les autres tables.

- **Vue 1**

- **Vue 2**
- **Vue 3**

Droits et Vues Comptable:

Le comptable vérifie l'argent qui rentre (les ventes) , l'argent qui sort (les factures des fournisseurs). Il ne peut que consulter les vues en relation avec le chiffre d'affaires, les dépenses et de la pharmacie. D'autre part, ils peuvent vérifier les remboursements à recevoir, c'est-à-dire la sommes remboursées par les mutuelles (et/ou l'Etat).

- **Vue 4**
- **Vue 5**
- **Vue 7**

Droits et Vues Gestionnaire des Commandes et lots (stock en général) :

Le gestionnaire des commandes peut accéder aux informations sur les lots pour suivre le stock disponible de chaque médicament. Il peut aussi consulter les informations sur les fournisseurs pour ainsi décider à qui envoyer les commandes mais il peut aussi consulter les lignes de ventes et les médicaments pour évaluer les ventes et anticiper les commandes. Enfin, il peut ajouter, modifier et supprimer des commandes.

- **Vue 2**
- **Vue 3**

Droits Administrateur et Vues de la base de donnée:

Possède un accès complet au système de gestion de la base de données et ses fonctionnalités dont modifier, changer et manipuler les rôles des utilisateurs.

Droits et Vues Client :

Un client a un accès en lecture seule à son historique d'achats au sein de la pharmacie, il peut consulter le nom des médicaments, la date et la facture associées à chacun de ses achats.

- **Vue 6**
-

Définitions des droits d'accès:

```

-- COMPTABLE
CREATE ROLE COMPTABLE;
GRANT SELECT ON VUE_CA_MENSUEL, Remboursement_Mutuelles, DEPENSE TO COMPTABLE;

--- CLIENT --- INCERTAIN
CREATE ROLE CLIENT;
GRANT SELECT ON HISTORIQUE TO CLIENT;

-- ADMIN
CREATE ROLE ADMINISTRATEUR
GRANT SELECT ANY TABLE, INSERT ANY TABLE, UPDATE ANY TABLE, DELETE ANY TABLE TO ADMINISTRATEUR;
GRANT CREATE USER, DROP USER, CREATE ROLE TO ADMINISTRATEUR;

-- Gestionnaire des Commandes

create role Gestionnaire_Commandes
grant select on Stock_risque,Catalogue_Stock_medicament to Gestionnaire_Commandes;
grant select,insert on LIGNEORDONNANCE,LIGNEVENTE to Gestionnaire_Commandes;
grant select on on medicament,lot,fournisseur to Gestionnaire_Commandes;

-- Pharmacien
create role Pharmacien
grant select on Stock_risque,Catalogue_Stock_medicament,Infos_ordonnance to Gestionnaire_Commandes;
grant select,insert on LIGNEORDONNANCE,LIGNEVENTE to Gestionnaire_Commandes;
grant select on on medicament,lot,fournisseur to Gestionnaire_Commandes;

```

Requêtes

I. Gestion des Stocks et Fournisseurs

1 - Quels sont les médicaments qui sont fournis simultanément par les fournisseurs 'PharmaDis' et 'HealthNord' ?

```
SELECT m.Nom, m.Prix_Public      -- FONCTIONNE
FROM MEDICAMENT m
JOIN LOT l ON l.Code_CIP = m.Code_CIP
WHERE l.Nom = 'PharmaDis'
```

INTERSECT

```
SELECT m.Nom, m.Prix_Public
FROM MEDICAMENT m
JOIN LOT l ON l.Code_CIP = m.Code_CIP
WHERE l.Nom = 'HealthNord';
```

2 - Quels sont les fournisseurs qui sont capables de fournir la totalité des médicaments référencés dans le catalogue ?

```
select f.Nom                                -- FONCTIONNE
from fournisseur f
join lot l on l.Nom = f.Nom
group by f.Nom
having count(distinct l.CODE_CIP) = (select count(*)
                                     from medicament);
```

5 - Quel est l'état actuel des stocks (nom et quantité totale) pour chaque médicament disponible ?

```

select m.nom,sum(l.Quantite) as quantite__med_dispo    -
from medicament m
join lot l on l.CODE_CIP = m.code_cip
group by m.nom
having sum(l.Quantite) > 0;

```

4 - Quels sont les médicaments qui n'ont jamais été prescrits par un médecin ?

```

select m.nom          -- FONCTIONNE
from medicament m
where not exists (
    select *
    from ligneordonnance l
    where m.code_cip = l.id_medicament
);

```

Quels sont les lots en stock qui sont encore utilisables (quantité positive et date de péremption non dépassée) ?

16-17 - Quel est le fournisseur auprès duquel la pharmacie a passé le plus grand nombre de commandes ?

-- 16. Lister le fournisseur auprès duquel la pharmacie s'est procurés le plus de médicament, ainsi que le nombre de médicament

```

SELECT F.NOM, COUNT(DISTINCT M.CODE_CIP) AS NOMBRE_MEDICAMENT_FOURNI -- FONCTIONNE
FROM FOURNISSEUR F
JOIN LOT L ON F.NOM = L.NOM
JOIN MEDICAMENT M ON L.CODE_CIP = M.CODE_CIP
GROUP BY F.NOM
HAVING COUNT(DISTINCT M.CODE_CIP) >= ALL (
    SELECT COUNT(DISTINCT M2.CODE_CIP)
    FROM FOURNISSEUR F2
    JOIN LOT L2 ON F2.NOM = L2.NOM
    JOIN MEDICAMENT M2 ON L2.CODE_CIP = M2.CODE_CIP
);

```

-- 17 : Requête qui retourne le fournisseur à qui la pharmacie a passé le plus de commandes, ainsi que le nombre de commandes passées

```

SELECT Nom, COUNT(id_Commande) AS nb_commandes --FONCTIONNE
FROM COMMANDE
GROUP BY Nom
ORDER BY nb_commandes DESC
FETCH FIRST 1 ROWS ONLY;

```

II. Performances et Traçabilité des Pharmaciens

3 - Quels sont les pharmaciens qui n'ont traité aucune ordonnance durant le mois de décembre ?

```

select p.nom, p.prenom          -- FONCTIONNE
from pharmacien p
where not exists (
    select 1
    from ligneordonnance lo
    where lo.id_RPPS = p.id_RPPS and extract(month from date_traitement) = 12
);

```

14 - Quel pharmacien a réalisé le plus grand nombre de ventes sur le mois en cours ?

```

SELECT p.Nom, p.Prenom, COUNT(v.id_Vente) AS nb_ventes -- FONCTIONNE
FROM PHARMACIEN p
JOIN VENTE v ON p.id_RPPS = v.id_Pharmacien
WHERE EXTRACT(MONTH FROM v.DateVente) = EXTRACT(MONTH FROM SYSDATE)
      AND EXTRACT(YEAR FROM v.DateVente) = EXTRACT(YEAR FROM SYSDATE)
GROUP BY p.id_RPPS, p.Nom, p.Prenom
HAVING COUNT(v.id_Vente) = (
    SELECT MAX(nb)
    FROM (
        SELECT COUNT(*) AS nb
        FROM VENTE v2
        JOIN PHARMACIEN p2 ON v2.id_Pharmacien = p2.id_RPPS
        WHERE EXTRACT(MONTH FROM v2.DateVente) = EXTRACT(MONTH FROM SYSDATE)
              AND EXTRACT(YEAR FROM v2.DateVente) = EXTRACT(YEAR FROM SYSDATE)
        GROUP BY p2.id_RPPS
    )
);

```

10 - Pour une ordonnance spécifique, quel est le pharmacien responsable de sa délivrance (traçabilité) ?

```

-- 10. Afficher, pour une ordonnance donnée, le pharmacien qui l'a traité, dans un objectif de traçabilité.

SELECT P.Id_RPPS, P.Nom, P.Prenom          -- FONCTIONNE
FROM PHARMACIEN P
JOIN LIGNEORDONNANCE LO ON LO.id_RPPS = P.id_RPPS
WHERE LO.id_Ordonnance = &id_ordonnance; -- on saisit l'ordonnance en question

```

9 - Lister tous les pharmaciens de l'équipe et afficher leurs ventes respectives (inclure ceux n'ayant fait aucune vente).

```

SELECT P.Nom, P.Prenom, V.id_Vente -- FONCTIONNE
FROM PHARMACIEN P
LEFT JOIN VENTE V ON V.id_Pharmacien = P.id_RPPS;

```

III. Statistiques de Vente

7 - Quel est le médicament le plus vendu en quantité (le "best-seller") ?

```
SELECT M.nom
FROM MEDICAMENT M
JOIN LOT L ON L.code_cip = M.code_cip
JOIN LIGNEVENTE LV ON LV.numero_de_lot = L.num_lot
GROUP BY M.nom
HAVING SUM(LV.quantité_vendu) >= ALL (
    SELECT SUM(LV2.quantité_vendu)
    FROM MEDICAMENT M2
    JOIN LOT L2 ON L2.code_cip = M2.code_cip
    JOIN LIGNEVENTE LV2 ON LV2.numero_de_lot = L2.num_lot
    GROUP BY M2.nom
);
```

8 - Quelle est la liste des 5 médicaments les plus vendus par la pharmacie ?

```
SELECT M.nom, SUM(LV.quantité_vendu) AS total_vendu -- FONCTIONNE
FROM MEDICAMENT M
JOIN LOT L ON L.code_cip = M.code_cip
JOIN LIGNEVENTE LV ON LV.numero_de_lot = L.num_lot
GROUP BY M.nom
ORDER BY total_vendu DESC
FETCH FIRST 5 ROWS ONLY;
```

15 - Lister les pharmaciens qui ont vendus tous les médicaments

```
select p.id_RPPS
from pharmacien p, vente v, lignevente lv, lot l
where v.id_Pharmacien = p.id_RPPS
and lv.id_Vente = v.id_Vente
and l.num_lot = lv.numero_de_lot
group by p.id_RPPS
having count(distinct l.CODE_CIP) = (select count(*)
from medicament);
```

11 - Quelle est la quantité totale vendue pour chaque médicament durant le mois de décembre

```

SELECT M.NOM, SUM(LV.quantité_vendu) quantité_totale
FROM MEDICAMENT M
JOIN LOT L ON L.CODE_CIP = M.CODE_CIP
JOIN LIGNEVENTE LV ON LV.numéro_de_lot = L.num_lot
JOIN VENTE V ON V.id_Vente = LV.id_Vente
WHERE EXTRACT(MONTH FROM V.datevente) = 12
GROUP BY M.NOM;

```

IV. Suivi des Clients et Remboursements

6 - Quel est le taux de remboursement d'un client spécifique (recherche par nom et prénom) ?

```

select c.Nom,c.Prenom,co.taux_de_remboursement      -- FONCTIONNE
from client c
join couverture co on c.Nom_mutuelle = co.Nom_mutuelle
where c.nom = '&nomclient' and c.prenom = '&prenomclient';

```

12 - Quels sont les clients bénéficiant d'un taux de remboursement supérieur à la moyenne de la clientèle ?

```

SELECT CL.NOM, CL.PRENOM      -- FONCTIONNE
FROM CLIENT CL
JOIN COUVERTURE C ON C.Nom_mutuelle = CL.Nom_mutuelle
WHERE C.taux_de_remboursement > (
    SELECT AVG(C1.taux_de_remboursement) AS moyenne
    FROM COUVERTURE C1
    JOIN CLIENT CL1 ON C1.Nom_mutuelle = CL1.Nom_mutuelle
);

```

13 - Quels sont les clients enregistrés auprès de la mutuelle "Malakoff Humanis" ?

```

SELECT CL.NSSI      -- FONCTIONNE
FROM CLIENT CL
WHERE CL.Nom_mutuelle = 'Malakoff Humanis';

```

19 - Combien de clients différents sont répertoriés dans la base de données ?

```
SELECT COUNT(DISTINCT(NSSI))
FROM client;
```

18 - Pour un client donné, quelles sont toutes les ordonnances qui lui ont été rattachées ?

```
SELECT id_Ordonnance,date_Prescription,date_De_Peremption
FROM Ordonnance o
JOIN client c ON c.NSSI = o.NSSI
WHERE c.Nom = &NomClient and c.Prenom = &PrénomClient;
```

20 - Quels clients ont acheté la totalité des médicaments figurant sur une de leurs ordonnances ?

```
SELECT o.id_ordonnance,c.nssi,c.nom,c.prenom --FONCTIONNE
FROM ordonnance o
JOIN client c ON o.nssi = c.nssi
JOIN ligneordonnance lo ON lo.id_ordonnance = o.id_ordonnance
JOIN lignevente lv ON lv.id_ordonnance = o.id_ordonnance
JOIN lot l ON lv.numero_de_lot = l.num_lot
WHERE l.code_cip = lo.id_medicament
GROUP BY o.id_ordonnance, c.nssi, c.nom, c.prenom
HAVING COUNT(DISTINCT lo.id_medicament) = COUNT(DISTINCT l.code_cip);
```

Metadonnées

- 1) Script : liste_ora_triggers.sql

```
SELECT
    TABLE_NAME AS "TABLES",
    TRIGGER_NAME AS "TRIGGERS",
    TRIGGER_TYPE AS "TYPE DU TIGGER",
    TRIGGERING_EVENT AS "DECLenchement"
FROM USER_TRIGGERS;
```

- 2) Script : liste_ora_constraints.sql

```
SELECT
    TABLE_NAME AS "TABLES",
    CONSTRAINT_NAME AS "NOM",
    CONSTRAINT_TYPE AS "TYPE DE LA CONTRAINTE",
    SEARCH_CONDITION AS "CORPS"
FROM USER_CONSTRAINTS
WHERE TABLE_NAME IN ( 'MEDICAMENT', 'MEDECIN',
                        'FOURNISSEUR', 'COUVERTURE', 'CLIENT',
                        'ORDONNANCE', 'VENTE', 'LOT', 'LIGNEVENTE' );
```

- 3) Script d'interrogation pertinents 1 : Evaluer le nombre de lignes pour chacune des tables

```
SELECT
    TABLE_NAME AS "TABLE",
    NUM_ROWS AS "LIGNES"
FROM USER_TABLES;
```

- 4) Script d'interrogation pertinents 2 : Afficher toutes les relations entre tables via les clés étrangères en listant la table fille, la table parent et le nom de la contrainte

```
SELECT
|   C.TABLE_NAME   AS TABLE_FILLE,
|   R.TABLE_NAME   AS TABLE_PARENT,
|   C.CONSTRAINT_NAME
FROM USER_CONSTRAINTS C
JOIN USER_CONSTRAINTS R
|   ON C.R_CONSTRAINT_NAME = R.CONSTRAINT_NAME
WHERE C.CONSTRAINT_TYPE = 'R';
```

Jeu de données :

Voir annexes